

RAG Retrieval Bug Fix

AI Music Discovery Assistant | Preference Extraction & Retrieval

Date: 2026-08-01

The Problem

The assistant extracted music preferences that did not exist in the dataset. For example, the LLM returned `{"genre": "Instrumental", "mood": "Relaxing"}`, while the dataset only contains lowercase values such as 'lofi', 'jazz', 'relaxed', and 'chill'. Because retrieval used an exact string match, these mismatched values returned zero songs and recommendation failed.

Root Cause

- **Unconstrained extraction:** the prompt never told the LLM which genres/moods were valid, so it invented plausible but non-existent categories.
- **Rigid matching:** the retriever required exact equality, so a near-miss like 'Relaxing' vs 'relaxed' filtered out every row.

The Solution (3 layers)

- **1. Grounded extraction.** The allowed genres and moods are read directly from `songs.csv` and injected into the prompt, so the LLM can only choose values that exist. This prevents hallucinated categories at the source.
- **2. Guard-clause ordering.** The 'no matching songs' check now runs immediately after retrieval, before any result is used - cleaner control flow.
- **3. Normalization safety net.** A new normalizer module snaps LLM output onto the dataset vocabulary: clean text -> exact match -> fuzzy fallback (difflib) -> empty. 'relaxing' becomes 'relaxed'; unknown values become "" so retrieval skips that filter instead of matching nothing.

Verified Behavior

Normalizer output confirmed by a standalone test:

Instrumental	/	Relaxing	->	''	/	relaxed	(fuzzy rescue + drop)
Rock	/	INTENSE	->	rock	/	intense	(case cleanup)
lofi	/	chill	->	lofi	/	chill	(exact, untouched)
''	/	happy	->	''	/	happy	(empty stays empty)

Files Changed

- **src/retriever.py** added `get_genres()` / `get_moods()`; skip empty filter values.
- **src/agent.py** inject allowed values into the prompt; normalize before retrieval; reorder empty guard.
- **src/normalizer.py** new module - pure, dependency-free (standard library only), easy to unit-test.

Why This Design

The fix treats the issue as schema alignment, not a prompt tweak. The dataset stays the single source of truth, so adding new songs or genres needs zero code changes. Prevention happens at the prompt; normalization is a defensive fallback layered underneath. Each concern lives in its own module,

keeping the code modular, testable, and easy to explain.

Code Changes

1. src/retriever.py - expose the dataset vocabulary

src/retriever.py

```
self.songs = pd.read_csv(csv_path)

+ def get_genres(self) -> list:
+     """Return the distinct genres available in the dataset."""
+     return sorted(self.songs['genre'].unique().tolist())
+
+ def get_moods(self) -> list:
+     """Return the distinct moods available in the dataset."""
+     return sorted(self.songs['mood'].unique().tolist())
```

2. src/retriever.py - skip empty filter values

src/retriever.py (retrieve)

```
- if "genre" in preferences:
+ if preferences.get("genre"):
    results = results[results['genre'].str.lower()
                      == preferences['genre'].lower()]

- if "mood" in preferences:
+ if preferences.get("mood"):
    results = results[results['mood'].str.lower()
                      == preferences['mood'].lower()]
```

3. src/agent.py - inject allowed values into the prompt

src/agent.py (recommend)

```
+ allowed_genres = self.retriever.get_genres()
+ allowed_moods = self.retriever.get_moods()

preference_prompt = f'''
Extract the user's music preferences.
+ The "genre" MUST be exactly one of: {allowed_genres}
+ The "mood" MUST be exactly one of: {allowed_moods}
+ Use the exact spelling and lowercase from the lists above.
+ If nothing fits, use an empty string "".
Return ONLY JSON. ...'''
```

4. src/agent.py - normalize + reorder empty guard

src/agent.py (recommend)

```
preferences = json.loads(preferences)
+ # Snap LLM output onto the dataset vocabulary.
+ preferences = normalize_preferences(
+     preferences, allowed_genres, allowed_moods)

songs = self.retriever.retrieve(preferences)
- valid_titles = songs['title'].tolist() # ran before guard
+ if songs.empty: # guard first
+     return "Sorry, I couldn't find any matching songs."
+ valid_titles = songs['title'].tolist()
```

5. src/normalizer.py - NEW module (core logic)

src/normalizer.py

```
+ import difflib
+
+ def normalize_value(value, allowed, cutoff=0.6):
+     if not value:
+         return ''
+     cleaned = value.strip().lower()
+     if cleaned in allowed: # 1. exact
+         return cleaned
+     matches = difflib.get_close_matches( # 2. fuzzy
+         cleaned, allowed, n=1, cutoff=cutoff)
+     return matches[0] if matches else '' # 3. drop
+
+ def normalize_preferences(prefs, genres, moods):
+     return {
+         'genre': normalize_value(prefs.get('genre', ''), genres),
+         'mood': normalize_value(prefs.get('mood', ''), moods),
+     }
```